

Scaling SSH Auth with Signed Certificates and CAs

Jesse Rodriguez
Principal Security Consultant
DirectDefense Inc.

Agenda

- **The problem** — why SSH keys stop scaling
- **The solution** — from distributed keys to centralized trust
- **Certificate signing options** — what gets set, what gets enforced
- **Live demos** — stand up a CA, sign a certificate, trust it and sign in, revoke access
- **Recap** — before vs. after, key takeaways, and resources

Authorized Keys Don't Scale

SSH key management at scale

- Every server keeps its own
`~/.ssh/authorized_keys`
- Revocation means finding and removing a key from every host it was ever added to
- A departing employee's key can sit on hundreds of systems
- Abandoned, stale keys never expire

Trust is scattered, revocation is manual, and nothing expires on its own.



Centralize Trust, Not Keys

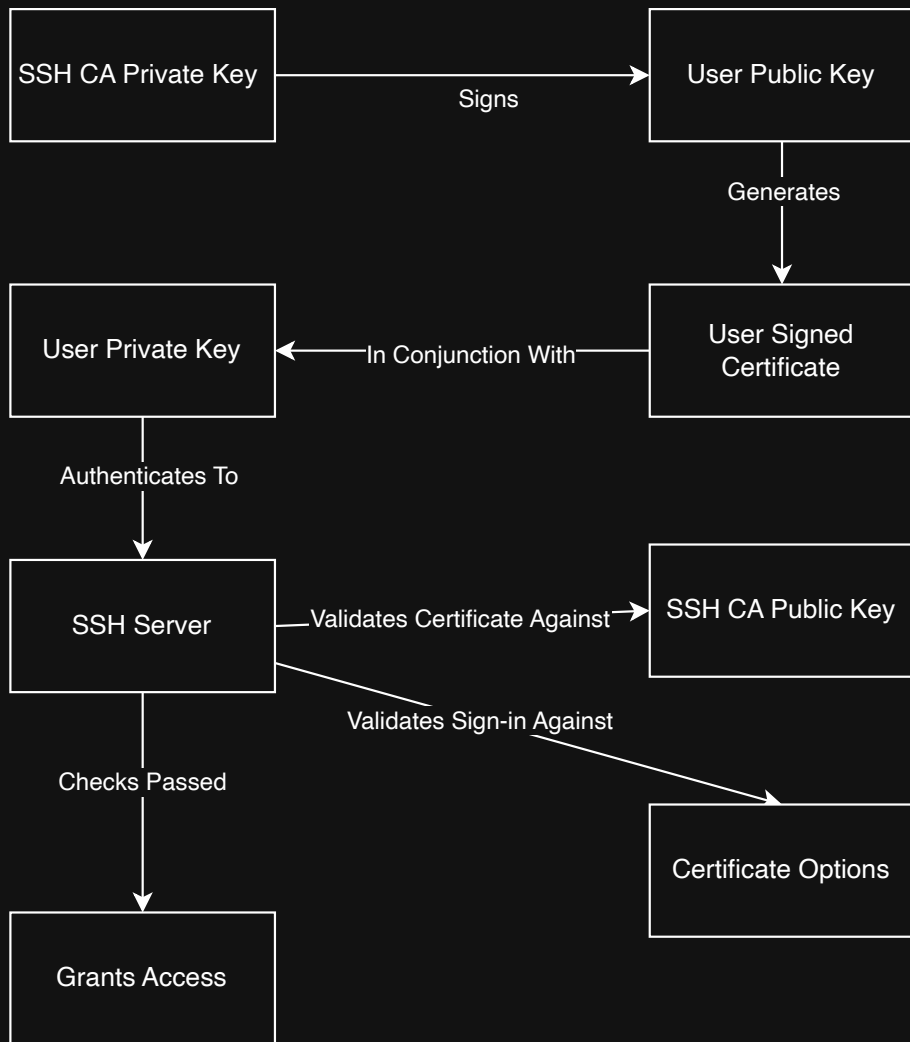
Instead of distributing public keys to every server, distribute one certificate.

- Stand up a **Certificate Authority** (CA) — just an SSH key pair
- The CA signs certificates for users and hosts
- `authorized_keys` files aren't needed



A distributed key-management problem becomes centralized.

HOW IT WORKS



Certificate Signing Options

Flags passed to `ssh-keygen -s` are enforceable by the server.

```
ssh-keygen -s ssh_user_ca_private_key \  
  -I "alice@example.com" \  
  -n alice,contractor \  
  -V +12h \  
  alice_ssh_key.pub
```

Flag	Sets	Enforced as
<code>-I</code>	Key ID	Identity string
<code>-n</code>	Principals	Which local accounts/roles the cert may log in as
<code>-V</code>	Validity	Time window

Live Demo 1

Standing up an SSH CA

Demo — Creating the Certificate Authority

```
# The CA is just an SSH key pair.  
# In production this key lives offline, in an HSM, etc.  
ssh-keygen -t ed25519 -f ssh_user_ca -C "DemoSSHCA"
```

```
>ls  
ssh_user_ca  ssh_user_ca.pub
```

Live Demo 2

Issuing and inspecting a certificate

Demo 2 — Sign a User Certificate

```
ssh-keygen -s ssh_user_ca -I "alice@example.com" -n alice,contractor -V +12h alice_ssh_key.pub
# -> produces alice_ssh_key-cert.pub
```

```
ssh-keygen -L -f alice_ssh_key-cert.pub
    Type: ssh-ed25519-cert-v01@openssh.com user certificate
    Public key: ED25519-CERT SHA256:JV5WlCPZiX9Wm2JZ7wC5tUjKqcsSFWB4Ftv4exFCRAI
    Signing CA: ED25519 SHA256:d1Ay340LbbwY0kxUkOqp0hQ/Mu0mwNk/emWj1Ry7UmA (using ssh-ed25519)
    KeyID: "alice@example.com"
    Serial: 0
    Valid: from 2026-09-15T16:00:00 to 2026-09-16T04:01:02
    Principals:
        alice
        contractor
    ...
```

Live Demo 3

Trusting the CA and signing in

Demo 3 — Sign In with a Certificate

```
# add directive to /etc/ssh/sshd_config
TrustedUserCAKeys /etc/ssh/ssh_user_ca.pub
#service ssh restart
```

```
ssh alice@98.89.34.222 -i alice_ssh_key
ssh contractor@98.89.34.222 -i alice_ssh_key
```

```
>journalctl -u ssh -f
sshd-session[2643]: Accepted publickey for alice from 98.89.34.222
  ssh2 ED25519-CERT SHA256:JV5WlCPZiX9Wm2JZ7wC5tUjKqcsSFWB4Ftv4exFCRAI
  ID alice@example.com (serial 0) CA ED25519 SHA256:d1Ay340LbbwY0kxUk0qp0hQ...

sshd-session[2695]: Accepted publickey for contractor from 98.89.34.222
  ssh2: ED25519-CERT SHA256:JV5WlCPZiX9Wm2JZ7wC5tUjKqcsSFWB4Ftv4exFCRAI
  ID alice@example.com (serial 0) CA ED25519 SHA256:d1Ay340LbbwY0kxUk0qp0hQ...
```

Live Demo 4

Revoking access

Demo 4 — Revoke a Certificate by Public Key

```
# create a KRL file from the certificate's public key
cat alice_ssh_key.pub >> /etc/ssh/revoked_keys
```

```
# add directive to /etc/ssh/sshd_config
RevokedKeys /etc/ssh/revoked_keys
#service ssh restart
```

```
ssh alice@98.89.34.222 -i alice_ssh_key
ssh contractor@98.89.34.222 -i alice_ssh_key
# Permission denied (publickey).
```

```
>journalctl -u ssh -f
error: Authentication key ED25519 SHA256:JV5WlCPZiX... revoked by file /etc/ssh/revoked_keys
error: Authentication key ED25519-CERT SHA256:JV5WlCPZiX... revoked by file /etc/ssh/revoked_keys
Connection closed by authenticating user alice 98.89.34.222 port 57012 [preauth]
error: Authentication key ED25519 SHA256:JV5WlCPZiX... revoked by file /etc/ssh/revoked_keys
error: Authentication key ED25519-CERT SHA256:JV5WlCPZiX... revoked by file /etc/ssh/revoked_keys
Connection closed by authenticating user contractor 98.89.34.222 port 57018 [preauth]
```

Recap

Before vs. After

	Key-Based (authorized_keys)	Certificate-Based (CA-signed)
Trust anchor	Every host's key file	One CA public key per host
Onboarding	Push key to every needed host	Issue a certificate
Offboarding	Find & remove key everywhere	Let it expire, or revoke it
Revocation	Manual edit, per host	One KRL, centrally distributed
Access control	Implicit, per system	Principals carried in the certificate
Audit trail	None, inferred from file state	Every login logged with identity & CA

Key Takeaways

- SSH key sprawl is a structural problem. It doesn't get better with more tooling around `authorized_keys`
- SSH certificates turn distributed trust into centralized trust
- One CA, one `TrustedUserCAKeys` line per host
- Principals carried in the certificate map straight to local accounts, no per-host account provisioning
- Short validity windows (`-V`) make expiration the default.
- Revocation is one KRL file away. No fleet-wide edits or waiting for expiry

Resources

- Certificate support ships with OpenSSH
 - <https://man7.org/linux/man-pages/man1/ssh-keygen.1.html#CERTIFICATES>
- Slide show created with
 - <https://sli.dev/>
- With help from
 - <https://claude.ai/>

Thank You

Questions?

Jesse Rodriguez

Principal Security Consultant

DirectDefense Inc.

jesse@directdefense.com

[linkedin.com/in/jrodriguez127](https://www.linkedin.com/in/jrodriguez127)

Discord/X: @ShadyBrownGuy

GitHub: @JRodriguez556

One More Thing

Automating revocation

```
#echo "* /10 * * * * root /path/to/key.revoker.checker.sh" >> /etc/crontab

#!/bin/bash
git clone https://gist.github.com/JRodriguez556/ac92a15e93893758377179d768babcc3
cd ac92a15e93893758377179d768babcc3/
git pull
grep -x -v -f /etc/ssh/revoked_keys revoked_key_demo > .revoked.tmp
cat .revoked.tmp >> /etc/ssh/revoked_keys
rm /home/*/ssh/authorized_keys
rm /root/.ssh/authorized_keys
```